

Session 3: Gravity & Collision & Locking

#1 Gravity

```
#define BLOCK_FALL_INTERVAL 500
```

블록이 떨어지는 시간 설정 (밀리초 단위)

```
static int last_fall_time = 0;
if (SDL_GetTicks() - last_fall_time > BLOCK_FALL_INTERVAL) {
if (!CheckCollision(currentPieceX, currentPieceY + 1, currentPieceType,
currentPieceRotation)) {
    currentPieceY++;
} else {
    LockPiece();                currentPieceX = 3;
    currentPieceY = 0;
    currentPieceRotation = rand() % 4;
    currentPieceType = rand() % 7;
    running = !CheckCollision(currentPieceX, currentPieceY,
currentPieceType, currentPieceRotation);
}
    last_fall_time = SDL_GetTicks();
}
```

메인 루프 안에 블록이 떨어진 시간을 추적하기 위해 last_fall_time 변수를 만들고

현재 시간과의 차이가 BLOCK_FALL_INTERVAL보다 크면 블록이 떨어짐(currentPieceY++)

만약 떨어질 위치에 블록이 들어갈 수 없다면 LockPiece()로 board에 고정하고 새로운 블록 생성하고 생성된 위치에도 들어갈 수 없다면 running == false 가 되어 게임이 종료되게 함.

#2 Collision Check

```
bool CheckCollision(int newX, int newY, int pType, int pRotation) {
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (shapes[pType][pRotation][i][j] == 1) {
                int boardX = newX + j;
                int boardY = newY + i;
                if (boardX < 0 || boardX >= COLUMNS || boardY < 0 || boardY >=
ROWS) return true;
                if (board[boardY][boardX] > 0) return true;
            }
        }
    }
}
```

```

    return false;
}

```

newX, newY, pType, pRotation 변수를 받아서 새로운 위치를 계산하고 각 위치가 board의 범위를 벗어나는지와 board가 이미 채워져 있는지 확인하여 충돌하는지 반환(충돌시 true, 아니면 false)

#3 Piece Lock

```

void LockPiece() {
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (shapes[currentPieceType][currentPieceRotation][i][j] == 1) {
                int boardX = currentPieceX + j;
                int boardY = currentPieceY + i;
                board[boardY][boardX] = currentPieceType + 1;
            }
        }
    }
}

```

전역변수로 되어있는 current piece에 관한 변수들로 블록의 위치를 계산하고 각 위치에 해당하는 board의 값 업데이트

#Optional Colors

SDL_pixels.h에 있는 SDL_Color 구조체 사용

```

const SDL_Color colors[8] = {
    {50, 50, 50, 255}, // blank
    {0, 255, 255, 255}, // I
    {0, 0, 255, 255}, // J
    {255, 128, 0, 255}, // L
    {255, 255, 0, 255}, // O
    {0, 255, 0, 255}, // S
    {255, 0, 255, 255}, // T
    {255, 0, 0, 255}, // Z
};

```

SDL_Color 정의, shape+1이 해당 블록 색깔과 맞게 함.

```

void DrawBlock(int pixelX, int pixelY, SDL_Color color) {
    // Set the color for the piece. For now, let's use a single color (e.g., Red).
    SDL_SetRenderDrawColor(renderer, color.r, color.g, color.b, color.a); // Bright Red (R, G, B, Alpha)
}

```

```

    // Define the rectangle for a single block on the screen
    SDL_Rect blockRect = {
        pixelX + 1,
        pixelY + 1,
        BLOCK_SIZE - 2, // Width of the block (slightly smaller for a grid
line effect)
        BLOCK_SIZE - 2 // Height of the block (slightly smaller for a grid
line effect)
    };

    // Draw the filled rectangle for this block
    SDL_RenderFillRect(renderer, &blockRect);
}

```

DrawBlock 함수에 SDL_Color를 받도록 추가하고, 받은 색상으로 그리게 설정
 그에 따라 DrawCurrentPiece 함수에서 DrawBlock 함수 호출하는 부분 수정함.
 (추가로 경계선 부분이 잘 보이게 하려고 블록 크기를 조금 줄임.)

```

void DrawBoard() {
    for (int row = 0; row < ROWS; row++) {
        for (int col = 0; col < COLUMNS; col++) {
            int pixelX = BOARD_OFFSET_X + col * BLOCK_SIZE;
            int pixelY = BOARD_OFFSET_Y + row * BLOCK_SIZE;
            DrawBlock(pixelX, pixelY, colors[board[row][col]]);
        }
    }
}

```

Board 를 그리기 위한 DrawBoard 함수를 만들고 메인 루프에 추가